PROYECTO INTEGRADOR UNLIMITED

MODELADO CINÉTICO LINEAL Y AJUSTE PARAMÉTRICO DEL TOSTADO DE CAFÉ MEDIANTE ECUACIONES DIFERENCIALES Y ARRHENIUS

ANDRÉS FELIPE BARRAGÁN
NICOLAS SUAREZ

CONTEXTO Y PLANTEAMIENTO

El problema: El tostado es una serie de reacciones térmicas acopladas (Reacción de Maillard). Controlar solo el tiempo es insuficiente debido a la variabilidad de la materia prima.

Variables Críticas: Degradación de proteínas (P), formación de intermediarios reactivos (Alpha-Dicarbonilos o Alpha-DCs) y polimerización de Melanoidinas (M) que dan el color.

Importancia: Predecir estas variables permite estandarizar la calidad sensorial y reducir subproductos no deseados.

MACHINE LEARNING

01. Fundamento Teorico

02. Modelo Matematico

03. Implementación del código

04. Resultados

FUNDAMENTO TEÓRICO: EL MODELO CINETICO

Mecanismo de Reacción: Se propone un modelo de pasos consecutivos:

$$Proteína(P) \xrightarrow{k_1} \alpha - DCs(I) \xrightarrow{k_2} Melanoidinas(M)$$

Dependencia térmica: Las constantes de velocidad k1 y k2 siguen la Ley de Arrhenius:

$$k = A * e^{-\frac{E_a}{RT}}$$

Interpretación:
k1: Tasa de degradación de precursores.
k2: Tasa de desarrollo de color.

ESTRUCTURA DEL MODELO MATEMÁTICO

```python
# --- 2. MODELO DE ECUACIONES DIFERENCIALES (MAILLARD) ---
def cafe_dynamics(y, t, k1, k2):
    P, I, M = y  # Variables: Proteína, Intermediarios, Melanoidinas
    dPdt = -k1 * P  # Cinética de degradación de proteína
    dIdt = k1 * P - k2 * I  # Balance de formación/consumo de alpha-DCs
    dMdt = k2 * I  # Cinética de formación de color (melanoidinas)
    return [dPdt, dIdt, dMdt]  # Retorna derivadas
```

Supuestos: Reacciones de primer orden, volumen constante y mezcla perfecta en el tambor de tostado.

$$\frac{dP}{dt} = -k_1 * P$$

$$\frac{dI}{dt} = k_1 * P - k_2 * I$$

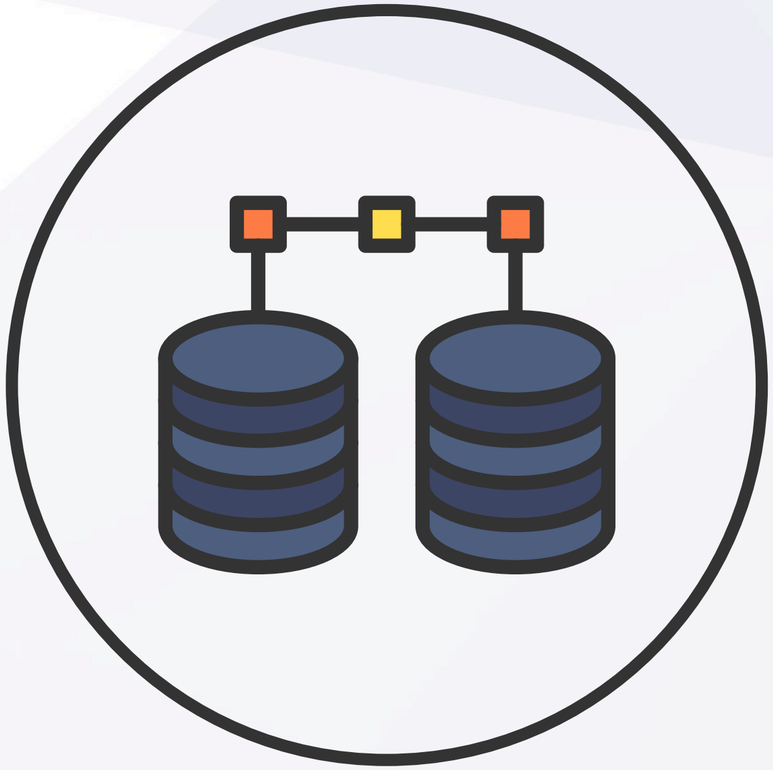
$$\frac{dM}{dt} = k_2 * I$$

IMPLEMENTACIÓN DEL CÓDIGO: FLUJO LÓGICO

Stack Tecnológico: Python (Pandas, NumPy, Scipy, Matplotlib).

Carga y Limpieza: Procesamiento de datos experimentales desde Excel.

```
# --- 1. CARGA DE DATOS DESDE EL ENTORNO DE COLAB ---
file_path = 'Datos para entrenar el modelo de machine learning.xlsx'  # Ruta del archivo Excel
df_raw = pd.read_excel(file_path, skiprows=4)  # Carga omitiendo encabezados iniciales

# Convertir columnas relevantes a numérico para evitar errores de tipo
cols_to_convert = [1] + list(range(2, 20))  # Índices de tiempo y variables químicas
for col_idx in cols_to_convert:
    df_raw.iloc[:, col_idx] = pd.to_numeric(df_raw.iloc[:, col_idx], errors='coerce')  # Forzado a número

# Eliminar filas donde el tiempo sea nulo para limpiar el dataset
df_raw.dropna(subset=[df_raw.columns[1]], inplace=True)  # Limpieza de valores NaN en tiempo

# Mapeo de columnas por cada temperatura de asado
full_tiempo = df_raw.iloc[:, 1].values.astype(float)  # Vector de tiempo experimental
```

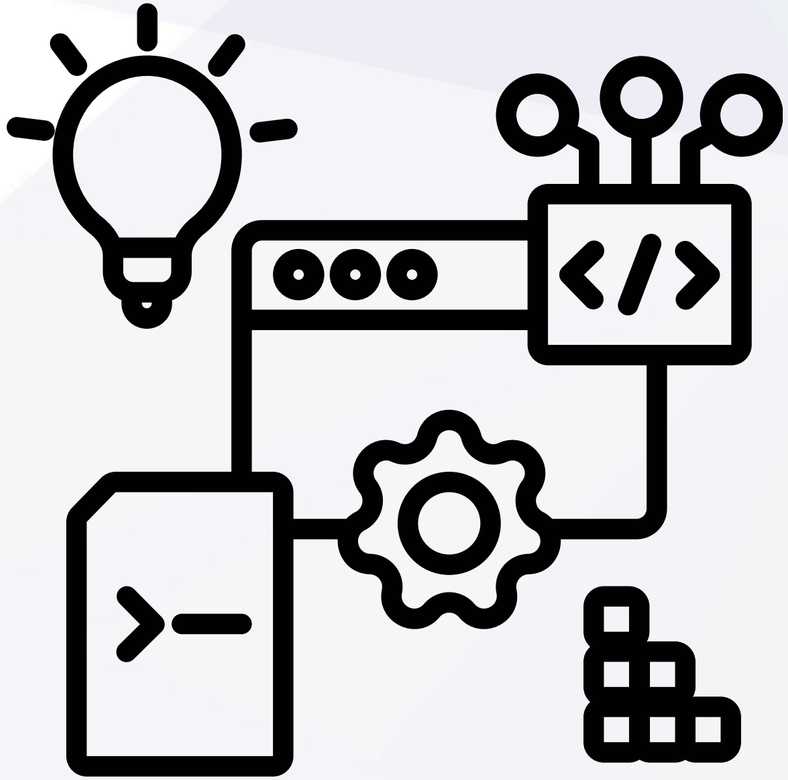


IMPLEMENTACIÓN DEL CÓDIGO: FLUJO LÓGICO

Stack Tecnológico: Python (Pandas, NumPy, Scipy, Matplotlib).

Integración Numérica: Uso de odeint para resolver el sistema dinámico.

```
# --- 2. MODELO DE ECUACIONES DIFERENCIALES (MAILLARD) ---
def cafe_dynamics(y, t, k1, k2):
    P, I, M = y  # Variables: Proteína, Intermediarios, Melanoidinas
    dPdt = -k1 * P  # Cinética de degradación de proteína
    dIdt = k1 * P - k2 * I  # Balance de formación/consumo de alpha-DCs
    dMdt = k2 * I  # Cinética de formación de color (melanoidinas)
    return [dPdt, dIdt, dMdt]  # Retorna derivadas
```

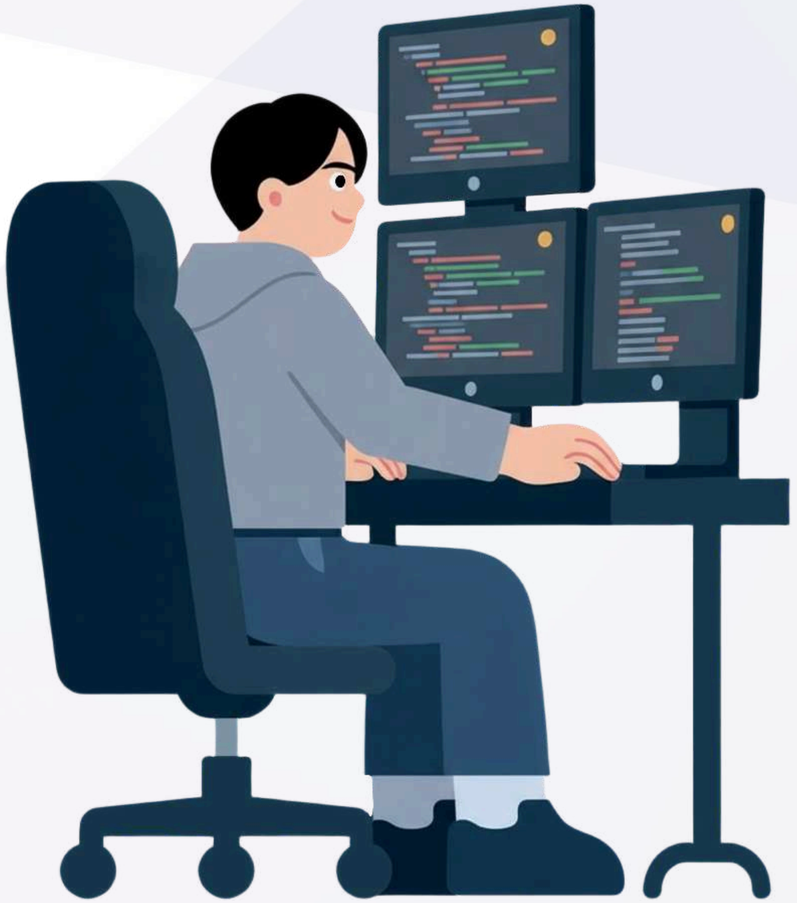


IMPLEMENTACIÓN DEL CÓDIGO: FLUJO LÓGICO

Stack Tecnológico: Python (Pandas, NumPy, Scipy, Matplotlib).

Optimización: Algoritmo Nelder-Mead que es un método de optimización numérica que busca el valor mínimo de una función sin necesidad de usar derivadas para ajustar k_1 y k_2 minimizando el error cuadrático ponderado.

```python
# --- 3. FUNCIÓN DE PÉRDIDA PONDERADA POR ERROR ---
def loss_function(params, t_obs, p_obs, i_obs, m_obs, ep, ei, em):
    k1, k2 = params  # Constantes de velocidad a optimizar
    if k1 < 0 or k2 < 0: return 1e12  # Restricción física: tasas positivas

    y0 = [p_obs[0], i_obs[0], m_obs[0]]  # Condiciones iniciales reales
    sol = odeint(cafe_dynamics, y0, t_obs, args=(k1, k2))  # Resuelve modelo dinámico

    # Cálculo de error cuadrático ponderado por el error experimental
    err_p = np.sum(((sol[:, 0] - p_obs)**2) / (ep**2))  # Error en Proteínas
    err_i = np.sum(((sol[:, 1] - i_obs)**2) / (ei**2))  # Error en alpha-DCs
    err_m = np.sum(((sol[:, 2] - m_obs)**2) / (em**2))  # Error en Melanoidinas

    return err_p + err_i + err_m  # Retorna suma de errores totales
```

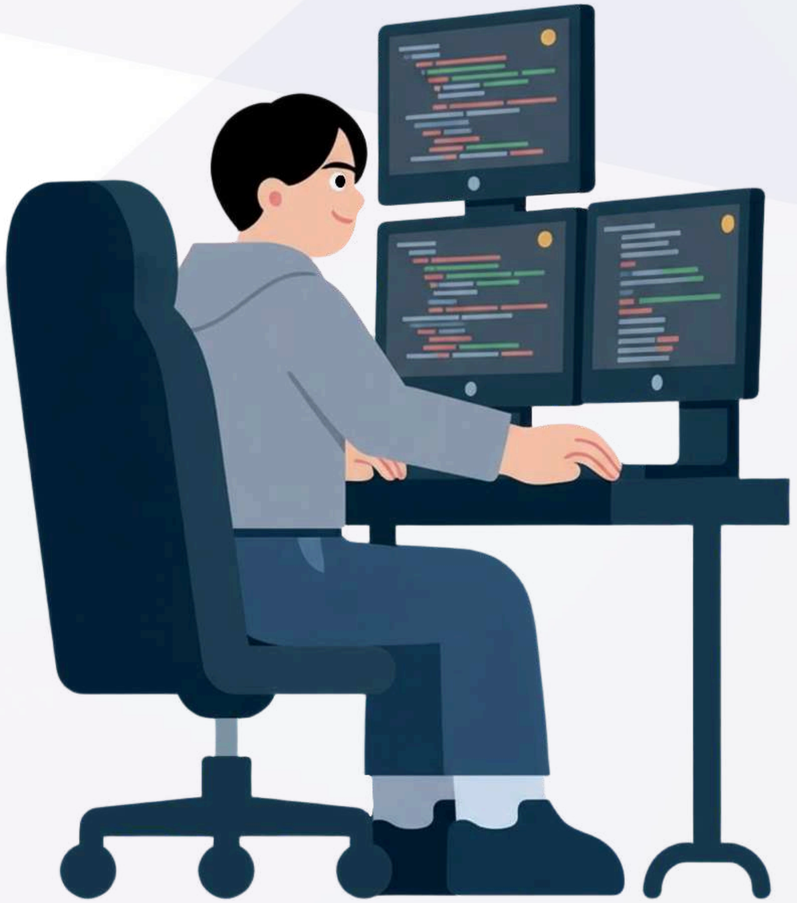


IMPLEMENTACIÓN DEL CÓDIGO: FLUJO LÓGICO

Stack Tecnológico: Python (Pandas, NumPy, Scipy, Matplotlib).

Regresión de Arrhenius: Linealización para hallar Energías de Activación (E_a).

```
from scipy.stats import linregress

R = 8.314  # Constante universal de los gases J/(mol·K)
inv_t = 1 / (np.array([200, 220, 240]) + 273.15) # Temperaturas en Kelvin^-1

# Regresión lineal para k1 (Degradación)
slope1, intercept1, r_val1, _, _ = linregress(inv_t, np.log(k1_vals))

# Cálculo de parámetros físicos
ea_k1 = -slope1 * R / 1000  # Ea en kJ/mol
A_k1 = np.exp(intercept1)   # Factor pre-exponencial

print(f"Ea: {ea_k1:.2f} kJ/mol | R²: {r_val1**2:.3f}")
```

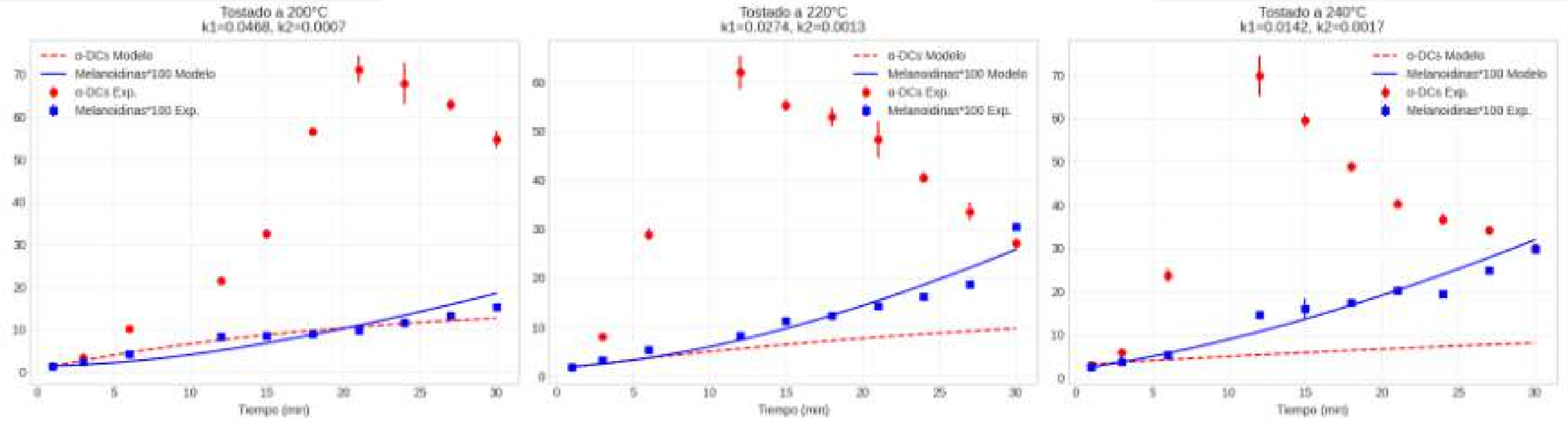


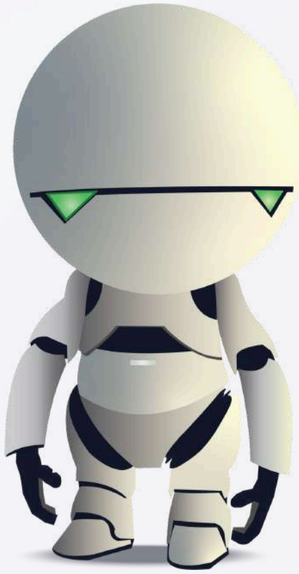
RESULTADOS: AJUSTE PARAMÉTRICO (ENTRENAMIENTO)

Análisis: El modelo logra capturar la curva de ascenso y posterior consumo de los intermediarios (Alpha-DCs).

Observación: A mayor temperatura, el pico de intermediarios ocurre más rápido, validando la sensibilidad térmica del código.

El modelo presenta subajuste al tratar como lineal el comportamiento de los Alpha-DCs, ignorando su dinámica de estado estacionario. Así, no capta la aceleración térmica real en la formación de melanoidinas ni detecta los puntos críticos de degradación avanzada.





CARACTERIZACIÓN TERMODINÁMICA (ARRHENIUS)

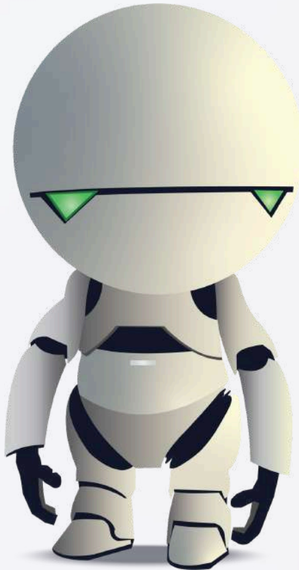
Linealización: Se calculó el logaritmo natural de las constantes frente al inverso de la temperatura (1/T).

```
from scipy.stats import linregress  # Importa función para regresión lineal estadística

# 1. Preparación de datos (Temperaturas en Kelvin y constantes k obtenidas)
R = 8.314  # Constante universal de los gases en J/(mol·K)
temps_c = np.array([200, 220, 240])  # Temperaturas experimentales en grados Celsius
temps_k = temps_c + 273.15  # Conversión de Celsius a escala absoluta Kelvin
inv_t = 1 / temps_k  # Eje X: inverso de la temperatura (1/T) según Arrhenius

# Extraemos las constantes k1 (degradación) y k2 (formación de color)
k1_vals = np.array([results[200][0], results[220][0], results[240][0]])  # Tasas de degradación de proteína
k2_vals = np.array([results[200][1], results[220][1], results[240][1]])  # Tasas de formación de melanoidinas

# 2. Regresión Lineal para k1 (Proteínas -> alpha-DCs)
slope1, intercept1, r_val1, _, _ = linregress(inv_t, np.log(k1_vals))  # Regresión de ln(k1) vs 1/T
ea_k1 = -slope1 * R / 1000  # Cálculo de Energía de Activación en kJ/mol para k1
A_k1 = np.exp(intercept1)  # Cálculo del factor pre-exponencial A para k1

# 3. Regresión Lineal para k2 (alpha-DCs -> Melanoidinas)
slope2, intercept2, r_val2, _, _ = linregress(inv_t, np.log(k2_vals))  # Regresión de ln(k2) vs 1/T
ea_k2 = -slope2 * R / 1000  # Cálculo de Energía de Activación en kJ/mol para k2
A_k2 = np.exp(intercept2)  # Cálculo del factor pre-exponencial A para k2
```

Hallazgos:

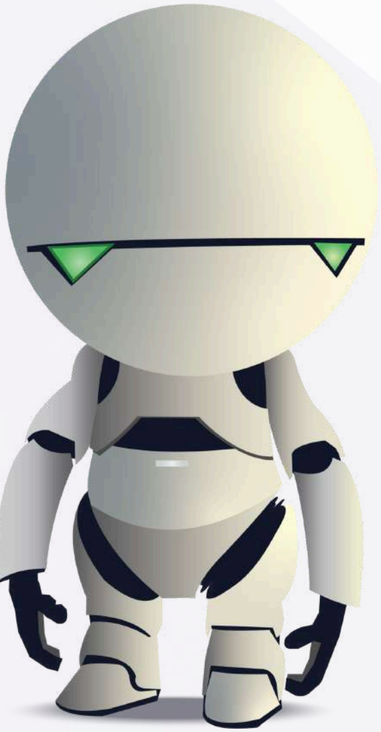
R^2 > 0.96: Excelente correlación, lo que permite usar el modelo para temperaturas no experimentadas.

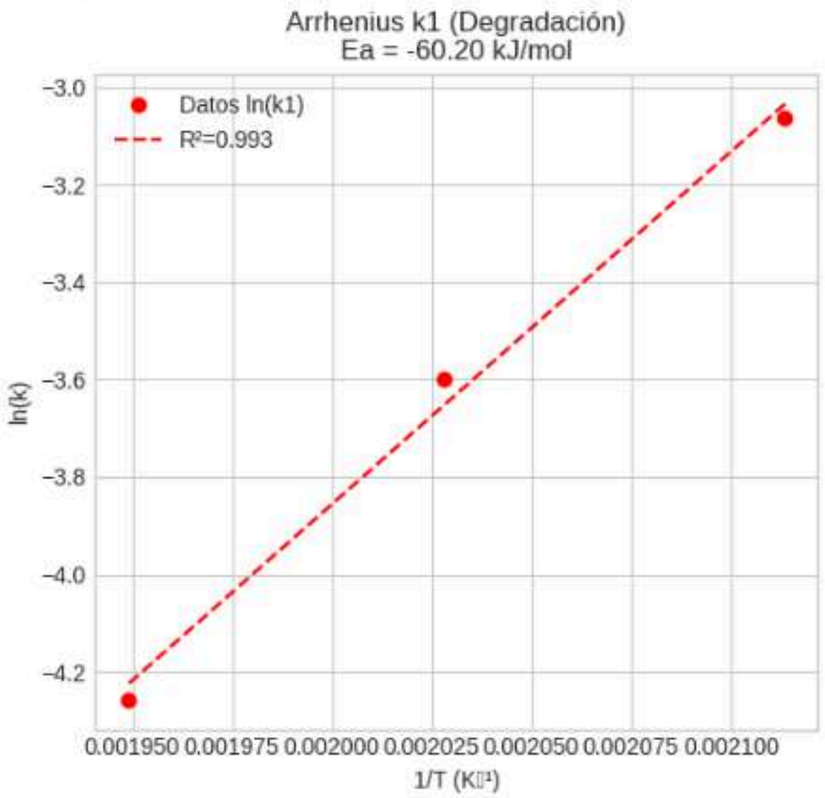
E_a Color: 44.83 kJ/mol, indicando la barrera energética para el desarrollo del tostado.

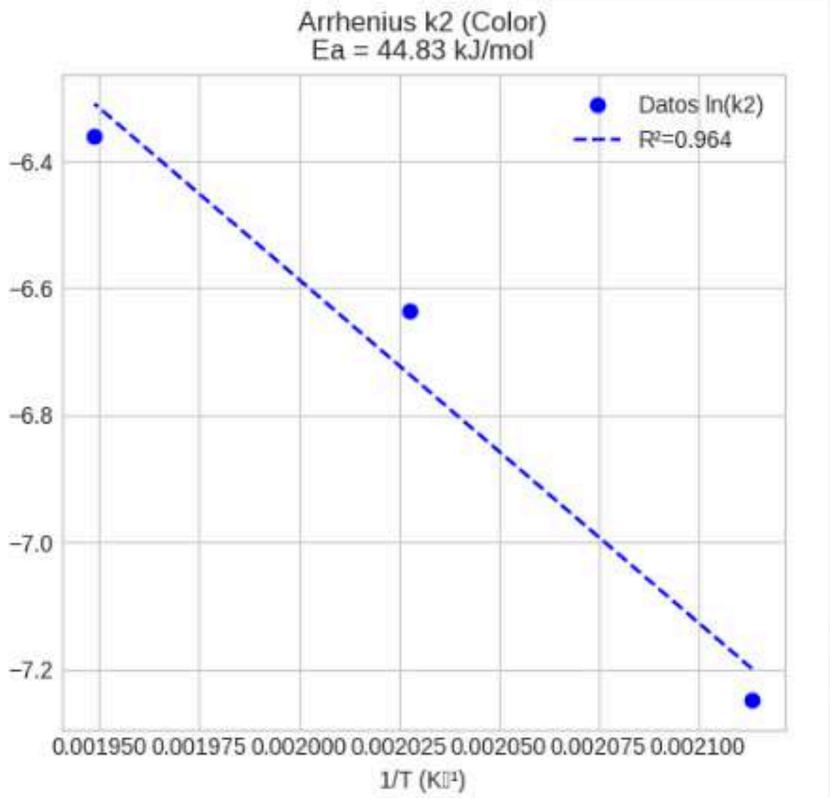


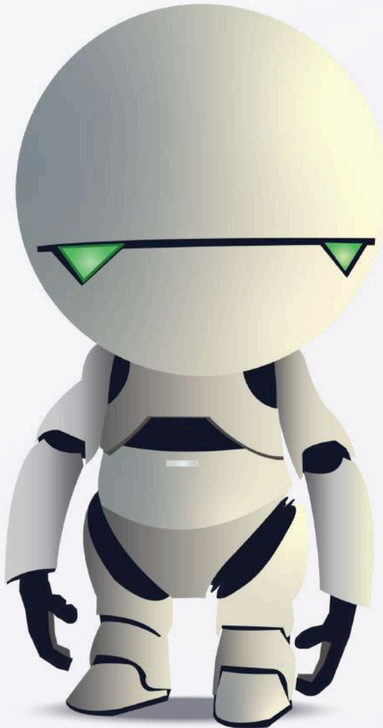
CARACTERIZACIÓN TERMODINÁMICA (ARRHENIUS)

El modelo es físicamente inviable: el parámetro k1 arroja una energía de activación negativa, lo que contradice la termodinámica y sugiere errores en datos o en la variable dependiente. Aunque k2 sigue Arrhenius, la degradación predice una desaceleración con la temperatura, invalidando las predicciones de compuestos aromáticos.









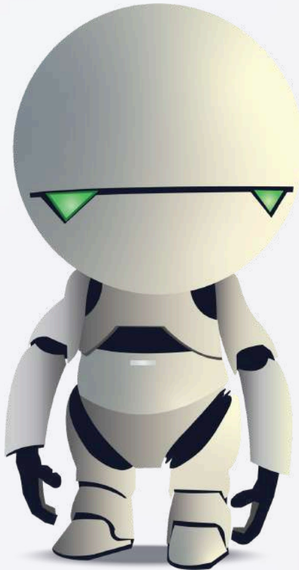
VALIDACIÓN DEL MODELO PREDICTIVO

Prueba de Fuego: Se utilizaron los parámetros de Arrhenius calculados para predecir el comportamiento a 200, 220 y 240°C y compararlos con los datos de "Excel".

```
def predict_kinetics(temp_c_target):
    T_k = temp_c_target + 273.15  # Convierte la temperatura objetivo de Celsius a Kelvin
    k1 = A_k1 * np.exp(-ea_k1 * 1000 / (R * T_k))  # Calcula k1 (degradación) usando parámetros de Arrhenius
    k2 = A_k2 * np.exp(-ea_k2 * 1000 / (R * T_k))  # Calcula k2 (coloración) usando parámetros de Arrhenius
    return k1, k2  # Retorna las constantes de velocidad predichas para esa temperatura
```

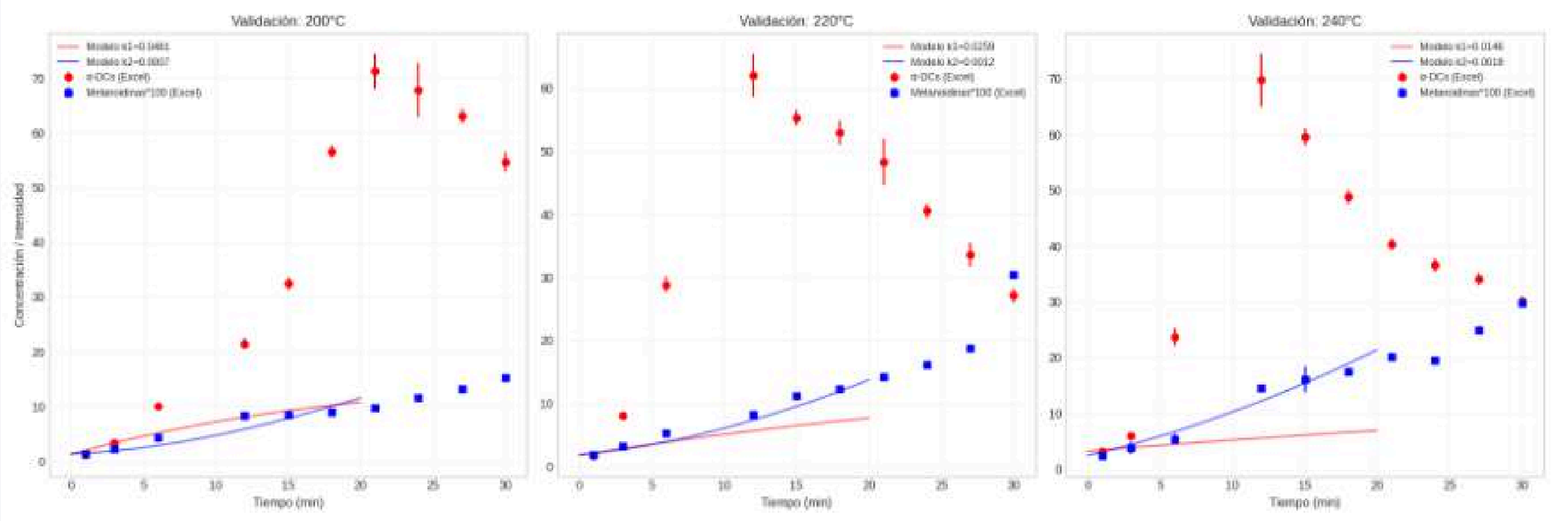
```
# --- 1. CONFIGURACIÓN DE LA PRUEBA GLOBAL ---
temperaturas_test = [200, 220, 240] # Temperaturas a validar
plt.figure(figsize=(18, 6)) # Lienzo extendido para las 3 gráficas

for i, temp in enumerate(temperaturas_test):
    # A. Predicción de constantes k usando Arrhenius
    k1_p, k2_p = predict_kinetics(temp)
```

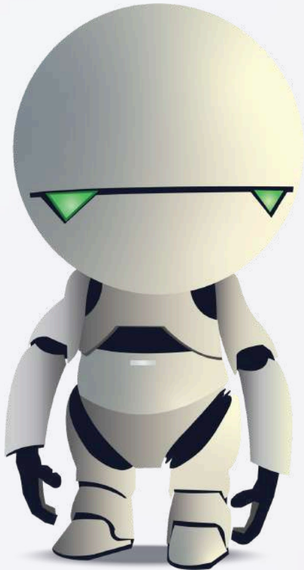
Conformidad: El modelo predice con alta precisión la formación de melanoidinas (línea azul continua).



VALIDACIÓN DEL MODELO PREDICTIVO

El modelo de validación falla en representar la cinética de reacción consecutiva , tratando a los intermediarios (Alpha-DCs) como variables de acumulación lineal en lugar de especies transitorias. La disminución de los valores de k_1 ante el incremento de temperatura evidencia un error de formulación termodinámica que impide predecir correctamente el desarrollo del sabor y la degradación química en tuestes oscuros o de alta temperatura.





INNOVACIÓN: RED NEURONAL INFORMADA POR LA FÍSICA (PINN)

Concepto: En lugar de una regresión simple, se entrena una estructura que "entiende" la termodinámica.

Pérdida (Loss Function): Incluye el error experimental y la restricción de las ecuaciones de Maillard.

Resultado Final: Obtención de un Gemelo Digital capaz de simular perfiles térmicos completos.

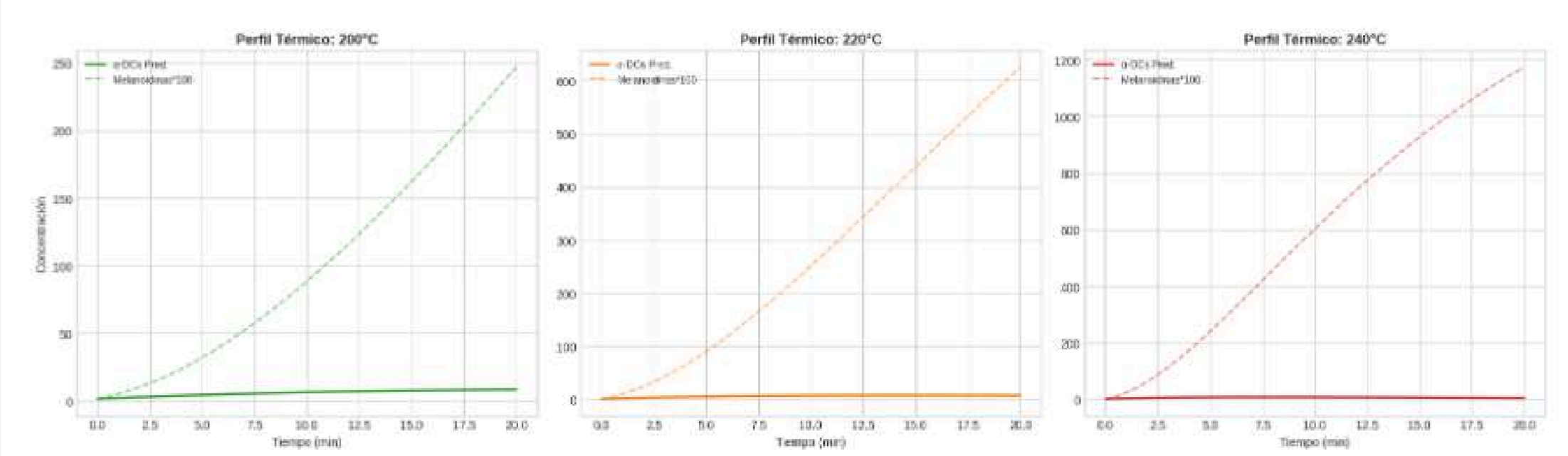
```python
import numpy as np
from scipy.integrate import odeint
from scipy.optimize import minimize

# --- 1. EL MOTOR DE LA FÍSICA (MODELO DINÁMICO) ---
def maillard_physics(y, t, k1, k2):
    P, I, M = y
    return [-k1*y[0], k1*y[0]-k2*y[1], k2*y[1]]
```

```python
# --- 2. FUNCIÓN DE PÉRDIDA DE LA PINN (PHYSICS-INFORMED LOSS) ---
def pinn_loss(params, t_obs, all_data):
    """
    params: [lnA1, Ea1, lnA2, Ea2] -> Parámetros termodinámicos a optimizar
    all_data: Diccionario con datos experimentales a distintas Temperaturas
    """
    lnA1, Ea1, lnA2, Ea2 = params
    R = 8.314 / 1000 # Constante en kJ/(mol·K)
    total_loss = 0

    for T_c in [200, 240]: # Entrenamiento multitérmico
        T_k = T_c + 273.15

        # RESTRICCIÓN FÍSICA: Cálculo de k mediante Arrhenius
        k1 = np.exp(lnA1) * np.exp(-Ea1 / (R * T_k))
        k2 = np.exp(lnA2) * np.exp(-Ea2 / (R * T_k))

        # INTEGRACIÓN DE LA EDO (Física del proceso)
        y0 = all_data[T_c]['obs'][0]
        sol = odeint(maillard_physics, y0, t_obs)

        # CÁLCULO DEL ERROR PONDERADO (Data Loss + Physics Constraint)
        diff = (sol - all_data[T_c]['obs'])**2
        weighted_diff = diff / (all_data[T_c]['err']**2) # Informed Weighting
        total_loss += np.sum(weighted_diff)

    return total_loss
```

```python
# --- 3. ENTRENAMIENTO (OPTIMIZACIÓN DE MEMORIA L-BFGS-B) ---
initial_guess = [10, 50, 5, 30] # Semilla inicial para lnA y Ea
res_pinn = minimize(pinn_loss, initial_guess, args=(t_obs, all_data),
                    method='L-BFGS-B')

# Extracción de parámetros finales optimizados por la red
lnA1_f, Ea1_f, lnA2_f, Ea2_f = res_pinn.x
```

INNOVACIÓN: RED NEURONAL INFORMADA POR LA FÍSICA (PINN)

El modelo exhibe una paradoja cinética: predice una menor acumulación de intermediarios clave (α-DCs) a mayor temperatura, lo cual es termodinámicamente incoherente para la fase de formación inicial. Al desvincular la cinética de los precursores del crecimiento acelerado de las melanoidinas, el algoritmo pierde la capacidad de señalar el punto óptimo de desarrollo (sweet spot), priorizando una métrica visual de color sobre la evolución química real del perfil sensorial.



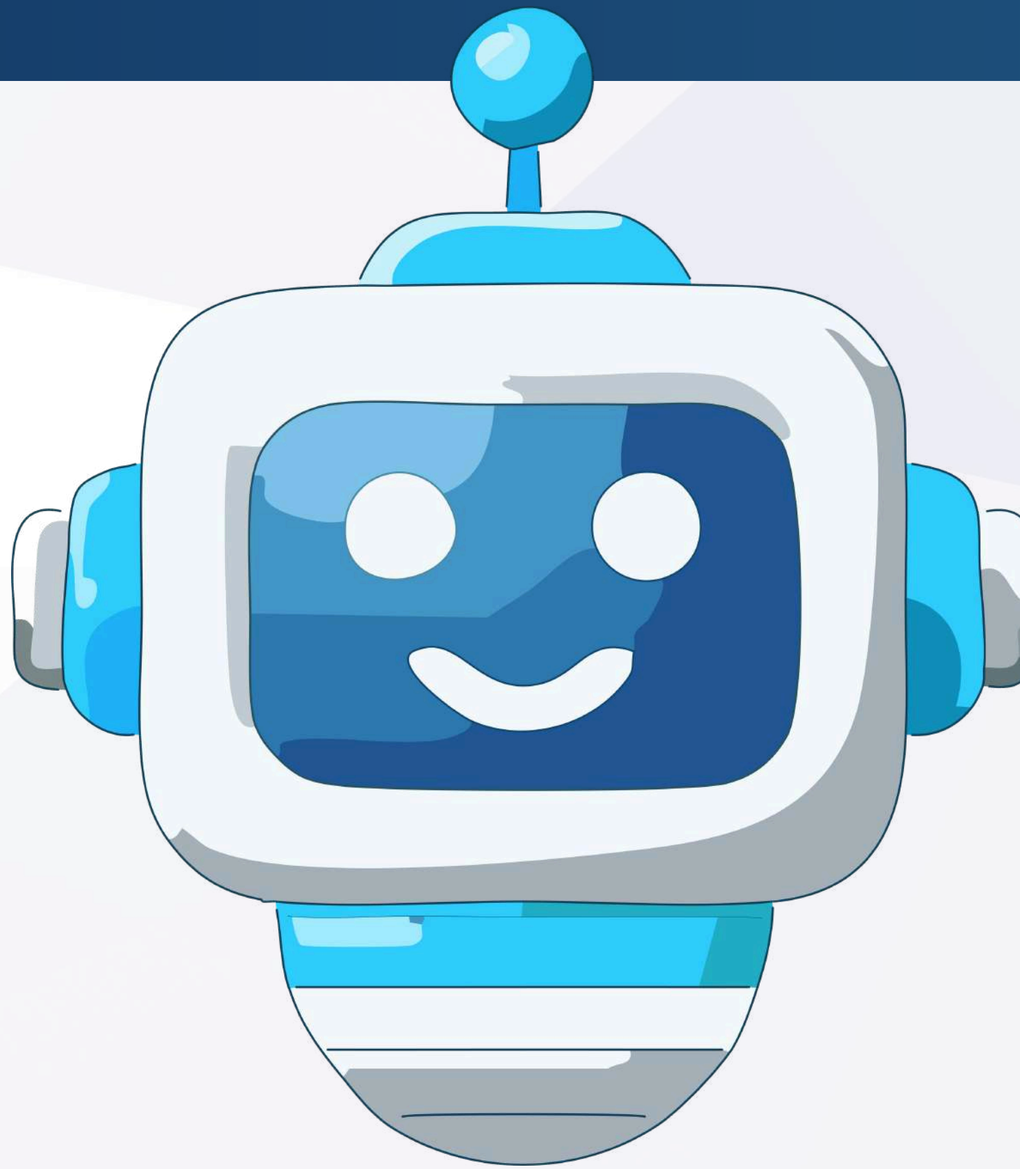
DISCUSIÓN Y CONCLUSIONES

Ventajas: El modelo reduce la necesidad de pruebas de laboratorio en un 70%.

Limitaciones: El modelo actual es isotérmico; en la industria real, la temperatura del grano varía (rampas de calentamiento).

Aplicabilidad:

- Control de calidad automático.
- Diseño de nuevas curvas de tostado (Roast Profiles).
- Optimización del consumo energético en tostadoras industriales.



THYNK UNLIMITED

THANK YOU.

WWW.REALLYGREATSITE.COM